Penetration Testing
A Hands-On Introduction to Hacking

by Georgia Weidman (No Starch Press, 2014)

Comprehensive Book Summary

Chapter-by-chapter overview with tools, techniques, and key takeaways

Prepared as a reading companion — not a substitute for the full text.

About This Summary

This document summarizes all 21 chapters (Chapter 0 through Chapter 20) of *Penetration Testing: A Hands-On Introduction to Hacking* by Georgia Weidman. The book teaches penetration testing end-to-end through a hands-on lab built around virtual machines (a Kali Linux attack box and Windows XP, Windows 7, and Ubuntu targets), following the standard phases of a penetration test: pre-engagement, information gathering, threat modeling, vulnerability analysis, exploitation, post exploitation, and reporting.

Each chapter section below includes: a short synopsis, the key techniques covered, and a box listing the specific **tools** introduced or used in that chapter. Tools that appear for the **first time** in the book are flagged in orange the first time they occur.

Book Structure (5 Parts)

- **Part I — The Basics** (Ch. 1–4): lab setup, Kali Linux, scripting, Metasploit fundamentals.
- **Part II — Assessments** (Ch. 5–7): information gathering, vulnerability discovery, traffic capture.
- **Part III — Attacks** (Ch. 8–15): exploitation, passwords, client-side attacks, social engineering, antivirus evasion, post exploitation, web app testing, wireless attacks.
- **Part IV — Exploit Development** (Ch. 16–19): buffer overflows on Linux and Windows, SEH overwrites, fuzzing and writing Metasploit modules.
- **Part V — Mobile Hacking** (Ch. 20): the Smartphone Pentest Framework (SPF).

Introduction Chapter

Chapter 0: Penetration Testing Primer

Chapter 0 defines penetration testing as going beyond a vulnerability scan: testers find weaknesses *and* exploit them to show real-world impact, simulating what an actual attacker could achieve.

It lays out the six classic phases of a pentest that structure the rest of the book: pre-engagement (scoping, rules of engagement, authorization/"get out of jail free" card, payment terms), information gathering (OSINT), threat modeling, vulnerability analysis, exploitation, post exploitation, and reporting (executive summary + technical report).

Key techniques & points

- Distinguishes a pentest (exploit to prove impact) from a vulnerability assessment (identify only).
- Pre-engagement must define scope, testing window, contacts, legal authorization, and NDA terms.
- A good report has an **executive summary** (background, posture, risk profile, recommendations, roadmap) and a **technical report** (detailed findings per phase).
- References the Penetration Testing Execution Standard (PTES) and NIST SP 800-115.

Tools used in this chapter

PTES methodology	NIST SP 800-115 guide		

Part I — The Basics

Chapter 1: Setting Up Your Virtual Lab

A fully hands-on chapter: the reader builds the lab used throughout the entire book. It walks through installing VMware (Player/Fusion/Workstation), downloading and customizing a Kali Linux attack VM, and configuring an isolated host-only virtual network so attacks stay contained.

It then builds three vulnerable targets: a Windows XP SP3 box loaded with vulnerable third-party software, a Windows 7 box, and an Ubuntu 8.10 box — each with static IPs, test accounts, and (for Windows XP) debugging tools for later exploit-development chapters.

Key techniques & points

- Kali Linux (successor to BackTrack) is the attack platform for the whole book.
- A host-only / isolated virtual network prevents lab attacks from leaking onto real networks.
- Windows XP target is deliberately loaded with vulnerable apps for exploitation practice.
- Immunity Debugger + Mona plugin are installed on Windows XP for later buffer-overflow chapters.

Tools used in this chapter

VMware (Player/Fusion/Workstation)	Kali Linux	Nessus (installed here, used in Ch.6)	Immunity Debugger
Mona.py plugin	Android emulator / SDK	Smartphone Pentest Framework (preview install)	

★ Newly introduced in this chapter: VMware, Kali Linux, Immunity Debugger, Mona.py

---

Chapter 2: Using Kali Linux

A Linux/command-line primer aimed at readers with little Linux background. Covers the filesystem, navigation, the man pages, user privilege management (useradd, sudoers, sudo), file permissions, text editing (vi), and core data-manipulation utilities.

Also introduces package management, process/service management, basic networking commands, Netcat as an all-purpose TCP/IP tool, and automating tasks with cron.

Key techniques & points

- grep, sed, and awk are presented as the core text-processing trio for parsing output.
- Netcat (nc) — opening a shell listener and pushing a reverse shell — is covered as a foundational skill reused throughout later exploitation chapters.
- cron jobs are introduced for automating recurring tasks (and later reused for Linux persistence).

Tools used in this chapter

Linux shell / Bash	man pages	useradd / sudo	vi editor
grep	sed	awk	apt / package manager
Netcat (nc)	cron		

★ Newly introduced in this chapter: Netcat (nc)

Chapter 3: Programming

A short, practical introduction to scripting for pentesters in three languages: Bash, Python, and C. The goal is to write small automation helpers (e.g., scripting pings/port checks) rather than full software engineering.

Covers variables, if-statements, and for-loops in Bash and Python, opening a TCP socket in Python, and compiling a simple C program — skills reused later for writing exploits and porting public exploit code.

Key techniques & points

- Bash scripting: ping sweeps, if-statements, for-loops to automate recon tasks.
- Python scripting: socket connections and conditionals — the language used for exploit prototyping later in the book (Part IV).
- Basic C compilation is introduced because many exploit targets and shellcode concepts are C-based.

Tools used in this chapter

Bash	Python	GCC / C compiler	

★ Newly introduced in this chapter: Python scripting

Chapter 4: Using the Metasploit Framework

The reader's first hands-on exploitation: using the MS08-067 Windows SMB vulnerability against the XP target via **msfconsole**. Covers searching/selecting modules, setting options (RHOST, RPORT, SMBPIPE, target), choosing payloads, and the difference between bind and reverse shells.

Introduces **Meterpreter** as Metasploit's advanced post-exploitation payload, the legacy **msfcli** single-line interface, **msfvenom** for building standalone payloads, the **multi/handler** module to catch callbacks, and running an auxiliary (non-exploit) module.

Key techniques & points

- Core msfconsole workflow: search → use → show options → set RHOST/payload → exploit.
- msfvenom replaced the older msfpayload/msfencode combo for standalone payload generation.
- multi/handler is the generic listener used whenever a payload is delivered outside of an auto-exploited module (web upload, email attachment, etc.).
- Auxiliary modules (e.g., scanner/smb/pipe_auditor) perform scanning/utility tasks, not exploitation.

Tools used in this chapter

Metasploit Framework (msfconsole)	Meterpreter	msfcli (legacy)	msfvenom
multi/handler module	Metasploit auxiliary modules	msfupdate	

★ Newly introduced in this chapter: Metasploit Framework, Meterpreter, msfvenom

---

Part II — Assessments

Chapter 5: Information Gathering

Opens Part II's simulated penetration test. Covers passive open-source intelligence (OSINT) gathering — Netcraft, Whois, DNS reconnaissance (nslookup, zone transfers), and searching for employee email addresses — then introduces **theHarvester** and **Maltego** for automated OSINT correlation.

Moves to active recon: manual port scanning concepts (TCP handshake, banner grabbing with Netcat) followed by automated scanning with **Nmap** (SYN scan, version detection, UDP scan).

Key techniques & points

- OSINT is free, passive, and low-risk — it should precede any active scanning.
- theHarvester automates searching search engines/PGP servers for emails and hosts.
- Maltego visually maps relationships between domains, people, and infrastructure.
- Nmap flags covered: -sS (SYN scan), -sV (version detection), -sU (UDP scan), -oA (output all formats).

Tools used in this chapter

Netcraft	Whois	nslookup / host (DNS recon)	theHarvester
Maltego	Netcat (banner grabbing)	Nmap	

★ Newly introduced in this chapter: theHarvester, Maltego, Nmap

---

Chapter 6: Finding Vulnerabilities

Covers the vulnerability-analysis phase: turning Nmap results into candidate vulnerabilities, running the **Nessus** vulnerability scanner (policies, scanning, interpreting rankings, exporting results), and using the **Nmap Scripting Engine (NSE)** for targeted checks.

Also covers Metasploit's own scanner and exploit "check" modules, web-app scanning with **Nikto**, probing default credentials on XAMPP/WebDAV, and manual analysis — e.g., using the VRFY SMTP command to enumerate valid usernames.

Key techniques & points

- Vulnerability scanners (Nessus) complement but don't replace manual critical thinking.
- NSE scripts (e.g., nfs-ls) extend Nmap to probe specific protocols/services.
- exploit "check" functions in Metasploit test exploitability without firing the full exploit.
- By the end, several concrete vulnerabilities are catalogued across the XP, Linux, and Win7 targets (MS08-067, phpMyAdmin, XAMPP/WebDAV default creds, NFS misconfiguration, vsftpd 2.3.4 backdoor, etc.).

Tools used in this chapter

Nessus	Nmap Scripting Engine (NSE)	Metasploit scanner/check modules	Nikto
cadaver (WebDAV client)	VRFY/SMTP enumeration		

★ Newly introduced in this chapter: Nessus (used), Nikto

Chapter 7: Capturing Traffic

Focuses on sniffing and manipulating network traffic. Introduces **Wireshark** for capturing, filtering, and following TCP streams, then covers **ARP cache poisoning** (concept, IP forwarding, and the **arpspoof** tool) to redirect switched-network traffic through the attacker machine, including impersonating the default gateway.

Extends to **DNS cache poisoning** with **dnsspoof**, SSL man-in-the-middle attacks using **Ettercap**, and defeating HTTPS redirects with **SSLstrip**.

Key techniques & points

- ARP poisoning exploits the lack of authentication in ARP to position as a man-in-the-middle.
- dnsspoof redirects victims to attacker-controlled sites by forging DNS responses.
- Ettercap automates SSL MITM; users still see a certificate warning unless combined with SSLstrip.
- SSLstrip downgrades HTTPS links to HTTP transparently, capturing credentials in plaintext.

Tools used in this chapter

Wireshark	arpspoof	dnsspoof	Ettercap
SSLstrip	iptables (traffic redirection)		

★ Newly introduced in this chapter: Wireshark, arpspoof, dnsspoof, Ettercap, SSLstrip

---

Part III — Attacks

Chapter 8: Exploitation

Converts the vulnerabilities found in Ch. 5–7 into actual compromises. Revisits MS08-067 with Meterpreter, exploits default WebDAV/XAMPP credentials to upload a PHP Meterpreter payload, and abuses an open phpMyAdmin panel to write files and a TFTP download.

Covers downloading sensitive files (configs, the Windows SAM), exploiting a buffer overflow in third-party software, exploiting vulnerable third-party web apps (TikiWiki), exploiting a backdoored service (vsftpd 2.3.4), and abusing an open/misconfigured NFS share to plant an SSH key for passwordless login.

Key techniques & points

- Shows multiple exploitation paths beyond "just run Metasploit": manual file upload, DB panel abuse, TFTP transfer, and NFS trust abuse.
- Demonstrates uploading custom msfvenom payloads (PHP Meterpreter) and catching them with handler.
- Backdoored open-source software (vsftpd) is treated as a real-world exploitation category.
- NFS misconfiguration + SSH key injection is a non-Metasploit, purely manual attack chain.

Tools used in this chapter

Metasploit (ms08_067_netapi)	msfvenom (PHP Meterpreter)	cadaver (WebDAV)	phpMyAdmin abuse
TFTP (atftpd)	TikiWiki exploit module	NFS mount + ssh-keygen	vsftpd backdoor

---

Chapter 9: Password Attacks

Covers both online and offline password attacks. Online: building custom wordlists with **CeWL** and **Crunch**, then brute-forcing services with **Hydra**.

Offline: dumping Windows password hashes (Meterpreter hashdump, bkhive/samdump2 from a mounted disk), explains **LM vs NTLM** hashing weaknesses, cracking with **John the Ripper** (Windows and Linux hashes, config-file passwords), rainbow tables, cloud cracking services, and dumping plaintext passwords from memory with **Windows Credential Editor (WCE)**.

Key techniques & points

- LM hashes are case-insensitive and split into two 7-character halves — far weaker than NTLM.
- John the Ripper supports both Windows (LM/NTLM) and Linux (shadow file) hash formats.
- Rainbow tables trade disk space for cracking speed via precomputed hash chains.
- WCE extracts plaintext credentials directly from memory on live Windows systems.

Tools used in this chapter

CeWL	Crunch	Hydra	Meterpreter hashdump
bkhive	samdump2	John the Ripper	Rainbow tables / RainbowCrack
Windows Credential Editor (WCE)			

★ Newly introduced in this chapter: Hydra, John the Ripper, WCE

Chapter 10: Client-Side Exploitation

Shifts from server-side to client-side attacks — software the *user* must open. Covers bypassing egress filters with all-port and HTTP/S Metasploit payloads, then a full browser exploitation chain (MS10-002 Aurora) including migrating sessions and AutoRunScript.

Covers malicious PDF exploits (Adobe utilprintf, PDF-embedded EXE), a then-zero-day Java exploit, the automated **browser_autopwn** module, and a Winamp-based client-side exploit.

Key techniques & points

- Client-side attacks require user interaction (opening a file/page) — the weak point is the human, foreshadowing Ch.11's social engineering.
- browser_autopwn fires many browser exploits automatically based on the visitor's fingerprint.
- Highlights how fully patched systems can still be compromised via zero-day client software bugs.
- Session migration (run migrate / AutoRunScript) is used to stabilize access after exploitation.

Tools used in this chapter

Metasploit browser exploits (ms10_002_aurora)	Adobe PDF exploits (utilprintf, pdf_embedded_exe)	Java exploit modules	browser_autopwn
Winamp exploit module			

★ Newly introduced in this chapter: browser_autopwn

Chapter 11: Social Engineering

Introduces the **Social-Engineer Toolkit (SET)** for automating human-targeted attacks: spear-phishing (crafting a malicious attachment, templating, single/mass email, setting up a listener), credential-harvesting web attacks (cloning a login page), mass email campaigns, and combining techniques into multipronged attacks.

Key techniques & points

- Social engineering targets the human element, which "cannot be patched."
- SET can also generate malicious USB sticks, QR codes, and rogue access points (mentioned, not walked through in depth).
- A multipronged attack might harvest credentials AND run a malicious Java applet simultaneously.

Tools used in this chapter

Social-Engineer Toolkit (SET)			

★ Newly introduced in this chapter: Social-Engineer Toolkit (SET)

Chapter 12: Bypassing Antivirus Applications

Addresses evading AV detection of Metasploit payloads. Starts with msfvenom encoders (shown to be largely ineffective against modern AV by themselves), then explains how AV engines work (signatures, heuristics) and tests payloads against **Microsoft Security Essentials** and **VirusTotal**.

Covers building a custom executable template and cross-compiling with MinGW, encrypting payloads with **Hyperion**, and finally evading AV more robustly with **Veil-Evasion** (VirtualAlloc injection + encryption).

Key techniques & points

- Simple msfvenom encoding (e.g., shikata_ga_nai alone) is generally insufficient against modern AV.
- VirusTotal is used to benchmark detection rates across many AV engines — with a privacy caveat (samples get shared with AV vendors).
- Hyperion encrypts an existing executable at runtime to defeat static signatures.
- Veil-Evasion was, at the time, a more advanced generator producing AV-evasive payloads.

Tools used in this chapter

msfvenom encoders	VirusTotal	MinGW cross-compiler	Hyperion
Veil-Evasion			

★ Newly introduced in this chapter: Hyperion, Veil-Evasion

Chapter 13: Post Exploitation

A broad tour of what to do after gaining access. Covers core Meterpreter usage (upload, getuid, other commands), Meterpreter scripts, Metasploit post-exploitation modules, and the **Railgun** API for calling Windows DLL functions directly from Ruby.

Covers local privilege escalation (getsystem, local exploits, UAC bypass, Linux udev escalation), local information gathering (file search, keylogging, credential harvesting, checking bash history), lateral movement (PSExec, pass-the-hash, SSHExec, token impersonation/Incognito, SMB capture), pivoting through compromised hosts (routes, pivot scanning, ProxyChains/Socks4a), and persistence (adding users, Metasploit persistence script, cron jobs).

Key techniques & points

- "The pentest truly begins after exploitation" — leveraging access is as important as gaining it.
- Pass-the-hash lets an attacker authenticate with a captured NTLM hash without cracking it.
- Pivoting (adding routes through a session) lets Kali attack hosts not directly reachable.
- Persistence mechanisms (new user, scheduled task/cron, Metasploit persistence script) maintain access.

Tools used in this chapter

Meterpreter (core + scripts)	Metasploit post modules	Railgun	getsystem
PSExec	Pass-the-hash	SSHExec	Incognito (token impersonation)
ProxyChains / Socks4a	Metasploit persistence script		

★ Newly introduced in this chapter: Railgun, Incognito, ProxyChains

---

Chapter 14: Web Application Testing

Targets a deliberately vulnerable sample web app. Covers intercepting/modifying requests with **Burp Suite (Burp Proxy)**, testing for and exploiting **SQL injection** (manual and with **SQLMap**), and XPath injection.

Covers Local and Remote File Inclusion (LFI/RFI), OS command execution via chained input, Cross-Site Scripting (XSS) — including leveraging a reflected XSS flaw with the **Browser Exploitation Framework (BeEF)** — Cross-Site Request Forgery (CSRF) concepts, and automated scanning with **w3af**.

Key techniques & points

- SQL injection in the sample app allows data extraction and even an OS command shell.
- A second injection flaw in XML-based login logic allows authentication bypass.
- BeEF "hooks" a victim's browser via XSS, giving the attacker a persistent foothold/control channel.
- Points to OWASP as the canonical ongoing resource for web application security.

Tools used in this chapter

Burp Suite (Burp Proxy)	SQLMap	Browser Exploitation Framework (BeEF)	w3af

★ Newly introduced in this chapter: Burp Suite, SQLMap, BeEF, w3af

---

Chapter 15: Wireless Attacks

Covers assessing Wi-Fi security. Sets up monitor mode and scans for access points, then attacks each major encryption scheme: open networks, **WEP** (explaining its weaknesses and cracking keys with **Aircrack-ng**), **WPA/WPA2** (Personal and Enterprise handshake flow, capturing and cracking the 4-way handshake), and **WPS** (its design flaw and brute-forcing the PIN with **Bully**).

Key techniques & points

- WEP's weak IV scheme makes it crackable by injecting/collecting enough packets.
- WPA/WPA2-Personal cracking requires capturing the 4-way handshake and brute-forcing/dictionary-attacking it.
- WPS's 8-digit PIN split into two halves makes it brute-forceable regardless of the Wi-Fi password strength.
- Underscores that wireless is frequently an overlooked perimeter weakness.

Tools used in this chapter

Aircrack-ng suite	Airodump-ng / Aireplay-ng	Bully (WPS)	

★ Newly introduced in this chapter: Aircrack-ng, Bully

Part IV — Exploit Development

Chapter 16: A Stack-Based Buffer Overflow in Linux

Opens Part IV with memory/stack theory (registers, stack frames, return addresses) before building a minimal custom Linux buffer-overflow exploit against a deliberately vulnerable C program using the unsafe strcpy

function.

Walks through crashing the program, debugging it with **GDB**, controlling EIP (the instruction pointer) by overflowing the buffer, hijacking execution to redirect to another function in the program, and a note on endianness.

Key techniques & points

- strcpy() with no bounds checking is the classic root cause of stack overflows.
- GDB is used to inspect registers/stack and pinpoint exactly where EIP gets overwritten.
- Hijacking EIP lets the attacker redirect execution anywhere in the process's address space.
- Establishes the foundational technique extended to real-world software in Ch.17–19.

Tools used in this chapter

GDB (GNU Debugger)	GCC (compiling the vulnerable target)		

★ Newly introduced in this chapter: GDB

Chapter 17: A Stack-Based Buffer Overflow in Windows

Applies Ch.16's concepts to a real, publicly known vulnerable Windows program: **War-FTP** has a buffer overflow in its Username field. Finds and verifies the crash, then precisely locates EIP using a cyclic/De Bruijn pattern and verifies the offset.

Instead of hardcoding a memory address, the exploit finds a **JMP ESP** instruction in a loaded DLL to reliably redirect execution into attacker-controlled memory, then fills it with shellcode generated by msfvenom to pop a shell.

Key techniques & points

- Cyclic pattern generation (pattern_create/pattern_offset, Mona) finds the exact offset to EIP.
- JMP ESP is a classic, reliable redirection technique that avoids hardcoded addresses breaking across different system builds.
- msfvenom-generated shellcode is dropped directly into the crafted exploit buffer.

Tools used in this chapter

Immunity Debugger	Mona.py	msfvenom (shellcode generation)	War-FTP (target app)

Chapter 18: Structured Exception Handler Overwrites

Covers a second major Windows exploitation technique for crashes that don't cleanly control EIP but do overwrite the **Structured Exception Handler (SEH)** chain — again using War-FTP.

Explains passing control to SEH, locating the attack string in memory, using a **POP POP RET** instruction sequence from a module not compiled with **SafeSEH**, using a short jump to reach shellcode, and choosing a working payload.

Key techniques & points

- NSEH sits reliably at ESP+8 whenever an SEH-based crash occurs — a key landmark for exploit writers.
- SafeSEH-protected modules can't be used for the POP POP RET gadget; unprotected modules must be found.
- A short jump bridges from the overwritten SEH pointer to the actual shellcode in the buffer.
- Notes that bad characters (e.g., \xEB) sometimes force alternate jump techniques.

Tools used in this chapter

Immunity Debugger	Mona.py	msfvenom	

Chapter 19: Fuzzing, Porting Exploits, and Metasploit Modules

Closes Part IV with practical exploit-development workflow skills: **fuzzing** a target (code review plus brute-force input mutation) to discover a crash in a trivial FTP server, then **porting a public exploit** (finding a new return address, replacing shellcode, editing for the local environment).

Walks through writing an original **Metasploit module** by adapting a similar existing exploit and porting Python exploit code for the 3Com TFTP long-transport-mode overflow. Ends with an overview of exploit **mitigation techniques**: stack cookies, ASLR, DEP, and mandatory code signing.

Key techniques & points

- Fuzzing = systematically feeding malformed/oversized input to find crashes without manual code review alone.
- Porting public exploits (new offsets/return addresses/shellcode) adapts code to the specific lab environment.
- Writing a Metasploit module packages a working exploit for reuse by the community/other pentesters.
- Modern mitigations (ASLR, DEP, stack cookies, code signing) are introduced as what real-world exploit developers must bypass beyond the book's simplified examples.

Tools used in this chapter

Custom Python fuzzer	GDB / Immunity Debugger	Metasploit module development framework	

Part V — Mobile Hacking

Chapter 20: Using the Smartphone Pentest Framework

Closes the book with mobile device security, built around the author's own tool, the **Smartphone Pentest Framework (SPF)**. Covers mobile attack vectors (SMS, NFC, QR codes), setting up SPF with an Android emulator, building and deploying the SPF Android app, and connecting the SPF server to the app.

Covers remote attacks (default SSH login on jailbroken iPhones), client-side mobile attacks (a WebKit browser shell, USSD remote-control codes), creating malicious SPF agents (backdooring legitimate APKs), and mobile post exploitation — information gathering, remote control, privilege escalation on Android, and pivoting attacks from a compromised phone (e.g., running Nmap and a War-FTP exploit from the Android device against the Windows XP target).

Key techniques & points

- Mobile devices expand the attack surface via unique vectors: SMS, NFC, QR codes, USSD codes.
- A compromised phone can itself become a pivot point into a corporate network (BYOD risk).
- SPF mirrors Metasploit's agent/handler model but tailored to smartphones.
- Ends the book's journey from "setting up a lab" to assessing an entirely new device class.

Tools used in this chapter

Smartphone Pentest Framework (SPF)	Android SDK / emulator	Nmap (from mobile pivot)	War-FTP exploit (reused via pivot)

★ Newly introduced in this chapter: Smartphone Pentest Framework (SPF)

Master Tool Index (by first appearance)

The table below lists every major tool covered in the book, in the order it is first introduced, with the chapter where it appears and its primary purpose.

Tool	Chapter(s)	Purpose
VMware	1	Virtualization platform hosting the whole lab
Kali Linux	1	Debian-based penetration testing distribution (attack platform)
Immunity Debugger + Mona.py	1, 17, 18	Windows debugger used for exploit development
Netcat (nc)	2	General-purpose TCP/IP shell/listener/file-transfer utility
Bash / Python / C	3	Scripting and programming for automation and exploits
Metasploit Framework / Meterpreter	4, 8, 10, 13	Core exploitation framework and advanced payload
msfvenom	4, 8, 12, 17	Standalone payload/shellcode generator
theHarvester	5	Automated OSINT (emails, hosts) gathering
Maltego	5	Visual OSINT / relationship-mapping tool
Nmap (+ NSE)	5, 6, 20	Port scanning and scripted vulnerability/service probing
Nessus	1, 6	Automated vulnerability scanner
Nikto	6	Web server vulnerability scanner
Wireshark	7	Packet capture and traffic analysis
arpspoof / dnsspoof	7	ARP and DNS cache poisoning for MITM traffic redirection
Ettercap	7	Automated MITM / SSL interception
SSLstrip	7	Downgrades HTTPS to HTTP to capture plaintext credentials
CeWL / Crunch	9	Custom wordlist generation
Hydra	9	Online password brute-forcing
John the Ripper	9	Offline password hash cracking
Windows Credential Editor (WCE)	9	Extracts plaintext credentials from Windows memory
browser_autopwn	10	Automated multi-exploit browser attack module
Social-Engineer Toolkit (SET)	11	Automates phishing / social-engineering attacks
VirusTotal	12	Multi-engine malware/AV detection check
Hyperion	12	Runtime executable encryption for AV evasion
Veil-Evasion	12	Advanced AV-evasive payload generator
Railgun / Incognito / ProxyChains	13	Post-exploitation: Windows API access, token theft, pivoting
Burp Suite	14	Web proxy for intercepting/manipulating HTTP(S) traffic
SQLMap	14	Automated SQL injection exploitation
BeEF	14	Browser Exploitation Framework (XSS hooking/control)

Tool	Chapter(s)	Purpose
w3af	14	Automated web application vulnerability scanner
Aircrack-ng / Bully	15	Wi-Fi (WEP/WPA/WPA2) and WPS key cracking
GDB	16	Linux debugger used to analyze stack overflows
Smartphone Pentest Framework (SPF)	17	Mobile device penetration testing framework (author's own tool)

Final Notes

This book (2014) is explicitly introductory and hands-on: it favors a guided lab over exhaustive theory, and the author repeatedly notes each topic (web app testing, AV evasion, post exploitation, exploit development, mobile hacking) "deserves its own book." The Resources appendix at the end of the book lists further reading per chapter (e.g., *Metasploit: The Penetration Tester's Guide*, *Hacking: The Art of Exploitation*, OWASP resources, Corelan exploit-writing tutorials).

Because the book is now over a decade old, several tools and version-specific details are dated (e.g., BackTrack/old Kali releases, msfcli, specific CVEs like MS08-067, Microsoft Security Essentials as the AV target, WEP/WPA2 details). The underlying **methodology and phase structure of a pentest remain current** and are the book's lasting value.